

5. Programación funcional

5.1. Fundamentos: la función *filter*

La programación funcional es un paradigma de programación en el que las funciones son las únicas estructuras para crear programas. Así, las funciones pueden asignarse a variables y pasarse como parámetros de otras funciones.

JavaScript soporta este paradigma de programación, de igual forma que soporta el paradigma de programación orientada a objetos. A lo largo de esta unidad hemos visto algunos ejemplos de programación funcional, como los *callbacks*. En este apartado profundizaremos algo más, concretamente en el apartado de **procesamiento de arrays**.

Es muy habitual que los programas trabajen con *arrays* para realizar modificaciones sobre ellos, como, por ejemplo:

- Realizar un **filtrado** para obtener un subconjunto de elementos que cumplan una condición.
- **Buscar** un determinado elemento.
- Modificar un *array* para **transformarlo** en otro con otras propiedades derivadas de las primeras.
- Realizar un **cálculo** con los elementos del *array*.
- Realizar **agrupaciones** de elementos del *array* en función de alguna condición.

Estas operaciones suelen realizarse de manera **iterativa**, utilizando **bucles que recorran todos los elementos del array**. Por ejemplo, para realizar un **filtrado** utilizaríamos un código parecido a este:

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15
let resultado = [];
for (let dato of datos) {
  if (dato < 15) {
    resultado.push(dato);
  }
}

console.log(resultado); // [14, 3]
```

El código anterior es sencillo, y estamos familiarizados con él. Sin embargo, la estructura utiliza un **for** en el que se modifica la variable **resultado** de manera indirecta: no hay una clara asignación a **resultado**, algo del tipo **resultado = VALOR_RESULTADO_DEL_FILTRADO**. A continuación, se muestra su **alternativa funcional**:

```
let datos = [14, 22, 16, 3, 35];

// Obtener los elementos menores que 15
let resultado = datos.filter(function (dato) {
  return dato < 15;
});

console.log(resultado); // [14, 3]
```

En este segundo caso sí que tenemos una asignación sobre la variable **resultado**, lo que facilita su comprensión. Quizá el hecho de tener una función anónima complica un poco la visualización si no estamos acostumbrados. Para aclararlo aún más, mostramos una tercera versión:

```
let datos = [14, 22, 16, 3, 35];

// Función que comprueba si un dato es menor que 15
// Se ha utilizado una función flecha en este caso
// Como solo tiene un parámetro, se pueden omitir los paréntesis
// Como solo tiene una línea en el cuerpo, se pueden omitir las llaves y el
`return`
// Devuelve true si se cumple la condición y false en caso contrario
let menorQue15 = dato => dato < 15;

// Obtener los elementos menores que 15
let resultado = datos.filter(menorQue15);

console.log(resultado); // [14, 3]
```

En este último código se ve claramente que **resultado** es el resultado de realizar una operación **filter** con una condición **menorQue15**. La función **menorQue15** es una función que devuelve únicamente **true** o **false** y que se pasa como parámetro a otra función. A este tipo de funciones se las denomina funciones predicado.

! IMPORTANTE

Las funciones flecha son muy adecuadas para crear funciones predicado, ya que normalmente estas funciones no suelen hacer uso de **this**. Por supuesto, también pueden utilizarse las funciones convencionales.

! IMPORTANTE

La función que se pasa como parámetro a **filter** debe aceptar tres parámetros: el elemento que se esté procesando (único parámetro obligatorio), la posición y el *array* original. Sin embargo, es muy habitual que se declare únicamente el primero, que corresponde al elemento del *array* que se está procesando.

A continuación, veremos otras funciones que incorpora JavaScript además de **filter** para trabajar con *arrays* de manera funcional.

5.2. Find

Devuelve el primer elemento que cumpla la condición pasada en la función. A continuación, se muestra un ejemplo con un `array` de objetos:

```
let usuarios = [
  {nombre: "David", apellido: "Pérez", nif: 1},
  {nombre: "Juan", apellido: "Martínez", nif: 2},
  {nombre: "José", apellido: "Fernández", nif: 3}
]

let usuarioConNif2 = usuarios.find(item => item.nif == 2);
console.log(usuarioConNif2); // {nombre: "Juan", apellido: "Martínez", nif: 2},
```

Como podemos ver, la función que se pasa como parámetro a `find` es de nuevo una función predicado, ya que devuelve únicamente `true` o `false`. Como en el caso de `filter`, dicha función puede aceptar tres parámetros [elemento, índice y `array`], aunque se suele utilizar solo con el primero.

5.3. ForEach

Permite ejecutar una función en cada uno de los elementos del `array`. Por ejemplo, si queremos ejecutar la función `alert` en cada elemento de un `array`, podemos ejecutar:

```
let datos = ["a", "b", "c", "d"];
datos.forEach(alert);
```

Como en el caso anterior, la función de `callback` acepta tres parámetros (elemento, posición y `array` original). Un ejemplo completo sería:

```
let datos = ["a", "b", "c", "d"];
datos.forEach(function(item, indice, array) {
  console.log(`Procesando el elemento ${item}, en la posición ${indice}.
  El array es ${array}.`);
});

// Resultado:
// Procesando el elemento a, en la posición 0. El array es a,b,c,d.
// Procesando el elemento b, en la posición 1. El array es a,b,c,d.
// Procesando el elemento c, en la posición 2. El array es a,b,c,d.
// Procesando el elemento d, en la posición 3. El array es a,b,c,d.
```

! IMPORTANTE

En este caso, la función que se pasa como parámetro no es una función predicado, ya que no devuelve `true/false`. Así, hablaríamos de una función de `callback` únicamente.

5.4. Map

Se utiliza para realizar transformaciones de *arrays*. A continuación, se muestra un ejemplo:

```
let numeros = [1, 3, 5, 7];

let dobleNumeros = numeros.map(function(numero) {
  return numero * 2;
});

// No se modifica el array original
console.log(numeros); // [1, 3, 5, 7]
console.log(dobleNumeros); // [2, 6, 10, 14]
```

Hay que tener cuidado al trabajar con *arrays* de objetos, ya que la función que se pasa a **map** puede realizar modificaciones sobre dichos objetos. Si esto ocurre, es posible que se devuelva un nuevo array, cuyos elementos apunten a los mismos objetos originales.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A05_01.js**, con el código de ejemplo.

El código anterior no es necesariamente malo: simplemente hay que tener en cuenta su funcionamiento. Si no nos importa modificar el *array* original [o es lo que pretendemos], entonces, perfecto; si lo que queremos es que **map** devuelva un *array* de objetos distintos, deberíamos cambiar el programa.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A05_02.js**, con el código de ejemplo.

Como en el caso anterior, la función de *callback* acepta tres parámetros, aunque es muy habitual que se declare únicamente el primero, que corresponde al elemento del *array* que se está procesando.

5.5. Reduce

La última función que veremos es la más compleja. Esta función se encarga de recorrer todos los elementos del *array* para generar un único valor. Se suele utilizar para realizar cálculos (totales, medias, etcétera) o agrupaciones de datos.



IMPORTANTE

reduce devuelve un único valor, pero puede ser un objeto o un *array*: no tiene por qué ser un tipo de datos sencillo.

reduce admite dos parámetros:

1. Una función de *callback*, que admite cuatro parámetros: el resultado de la ejecución anterior del *callback* (habitualmente llamado **acumulador**) y los tres que conocemos de las funciones que hemos estudiado antes: el valor actual del *array*, el índice del elemento que se está procesando y el *array*. Únicamente son obligatorios los dos primeros.
2. Un **valor inicial**, que se utilizará en la primera ejecución del *callback* (ya que la primera vez que se ejecuta el *callback* no hay un resultado de la ejecución anterior). Este parámetro es opcional, aunque es muy habitual que se proporcione con un valor vacío: `""`, `0`, `[]` o `{}`.

Veamos un ejemplo de funcionamiento para calcular la suma total la media de los elementos de un *array*:

```
let numeros = [2, 5, 4];

let total = numeros.reduce(
  // Primer parámetro: función de callback
  function(acc, numero) {
    return acc + numero;
  },
  // Segundo parámetro: valor inicial
  0);

console.log(total); // 11
```

Los pasos de ejecución serían los siguientes:

1. **acc** = 0 (valor inicial). La función devuelve $0 + 2 = 2$.
2. **acc** = 2 (resultado de la ejecución anterior). La función devuelve $2 + 5 = 7$.
3. **acc** = 7 (resultado de la ejecución anterior). La función devuelve $7 + 4 = 11$.

Por último, veamos un ejemplo más complejo para realizar una **agrupación**, que consiste en clasificar un listado de coches por color. El resultado se devolverá en forma de **objeto**, y sus propiedades serán los colores de los coches existentes.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U02_A05_03.js**, con el código de ejemplo.

Los pasos de ejecución serían los siguientes:

1. **acc** = {} (valor inicial). La función comprueba si existe **acc["rojo"]**. Como no existe, lo crea mediante **acc["rojo"] = {cuenta: 0, listado: []}**. A continuación, incrementa **acc["rojo"].cuenta** y añade el primer objeto coche a **acc["rojo"].listado**. Por último, devuelve **acc**.
2. **acc** es un objeto con una propiedad definida, "rojo". La función comprueba si existe **acc["verde"]**. Como no existe, lo crea mediante **acc["verde"] = {cuenta: 0, listado: []}**. A continuación, incrementa **acc["verde"].cuenta** y añade el segundo objeto coche a **acc["verde"].listado**. Por último, devuelve **acc**.
3. **acc** es un objeto con dos propiedades definidas, "rojo" y "verde". La función comprueba si existe **acc["azul"]**. Como no existe, lo crea mediante **acc["azul"] = {cuenta: 0, listado: []}**. A continuación, incrementa **acc["azul"].cuenta** y añade el tercer objeto coche a **acc["azul"].listado**. Por último, devuelve **acc**.
4. **acc** es un objeto con tres propiedades definidas, "rojo", "verde" y "azul". La función comprueba si existe **acc["rojo"]**. Como si existe, utiliza el objeto existente mediante **acc["rojo"] = acc["rojo"]**. A continuación, incrementa **acc["rojo"].cuenta** (que en este caso vale 1, no 0, porque ya se había incrementado en el paso 1, y añade el cuarto objeto coche a **acc["rojo"].listado** (que ya tenía incluido el primer coche). Por último, devuelve **acc**.



IMPORTANTE

La función **reduce** es la más potente de todas las funciones que hemos estudiado en este apartado. Tanto es así que se pueden implementar todas las demás a partir de ella.